# ТЕМА 7. HTML И ТЕХНОЛОГИИ

## ЛЕКЦИЯ 7.11. HTML и JS. Введение

*Введение в сценарии JavaScript. Подключение сценариев. Синтаксис JavaScript. Переменные и константы*

## Курс Программирование в JavaScript

## Тема 1. Введение в JavaScript Урок 1. Введение в сценарии JS

## Введение

**JavaScript** — это кроссплатформенный, объектно-ориентированный скриптовый язык, который позволяет веб-разработчику управлять тем, как **ведет** себя веб-страница. Это делает JavaScript принципиально отличным от HTML, который отвечает за **структуру** веб-документа, и CSS, который формирует внешний **вид** веб-страниц.

Программы, написанные на языке JavaScript, называются **скриптами** или **сценариями**. Скрипты JavaScript могут быть вставлены в любое место HTML-документа, и как только браузер отображая код страницы доходит до скрипта – выполняются. Процесс выполнения скрипта называют **интерпретацией**.

**Важно!** Если **PHP** скрипт обрабатывается **на стороне сервера** с помощью PHP интерпретатора, то **JavaScript** выполняется в браузере пользователя **JavaScript интерпретатором**.

Сегодня каждый браузер поддерживает JavaScript, тем самым делая его языком веб-разработки. Вот некоторые из задач, которые может решать программист с помощью JavaScript:


©Pechora-PRO

- проверять правильность заполнения пользовательских HTML-форм;
- менять стили HTML-элементов, скрывать, показывать элементы и т.п.;
- отображать всплывающие и диалоговые окна;
- реагировать на действия посетителя, обрабатывать клики мыши, перемещения курсора и т.п.;
- посылать запросы на сервер и загружать данные без перезагрузки страницы.

## Подключение сценариев

Чтобы программа (сценарий) JavaScript запустилась, ее нужно внедрить в HTML-документ. Сценарии можно внедрить в HTML-документ различными стандартными способами:

- поместить код между открывающим и закрывающим тегами **script**;
- поместить скрипты во **внешний файл** (с расширением .js), а затем подключить его к документу HTML;
- поместить код непосредственно в **атрибут события** HTML-элемента.

Как уже было сказано, браузер читает HTML-документ сверху вниз и, начинает отображать страницу, показывая часть документа до тега script. Встретив тег script, переключается в JavaScript-режим и выполняет сценарий. Закончив выполнение, возвращается обратно в HTML-режим и отображает оставшуюся часть документа.

Если на странице используется много скриптов JavaScript, то могут возникнуть длительные задержки при загрузке, в течение которых пользователь видит пустое окно браузера. Одним из способов решения проблемы считается указание JS-сценариев после контента страницы, перед закрывающим тегом body.


©Pechora-PRO

Такое расположение сценариев позволяет браузеру загружать страницу быстрее, так как сначала загрузится контент страницы, а потом будет загружаться код сценария.

Для пользователей это предпочтительнее, потому что страница полностью визуализируется в браузере до обработки JavaScript-кода.

## Синтаксис JavaScript

Синтаксис JavaScript – это набор правил, по которым создаются программы JavaScript.

При написании программ на JavaScript используется набор символов **Unicode**, которая поддерживает практически все письменные языки, имеющиеся на планете.

Интерпретатор JavaScript игнорирует все пробельные символы (табуляции, пробелы, неразрывные пробелы), которые могут присутствовать между языковыми конструкциями и воспринимает текст программы как сплошной поток кода.

Кроме того, JavaScript также, по большей части, игнорирует символы перевода строки. Поэтому пробелы и символы перевода строки могут без ограничений использоваться в исходных текстах программ для форматирования и придания им удобочитаемого внешнего вида.

### Точка с запятой

Программа (сценарий) на языке JavaScript представляет собой перечень инструкций, которые выполняются веб-браузером.

Если несколько инструкций располагаются на одной строке, то между ними следует поставить знак "точка с запятой" (;).

```
<script>

    alert("Сообщение"); break;

</script>
```


©Pechora-PRO

Если каждая инструкция размещается на отдельной строке, то разделитель можно не писать. В таких случаях JavaScript интерпретирует переход на новую строчку как разделитель команд для автоматического ввода точек с запятой, завершающих инструкции.

Одна инструкция может располагаться на нескольких строчках:

```
<script>

    var user = "Name";

    alert("Рады вас видеть, " +

    user +

    " !");

</script>
```

В этом случае JavaScript ждёт завершение выражения и поэтому автоматически не вставляет виртуальную точку с запятой между строчками.

Тем не менее, рекомендуется всегда добавлять точки с запятой – это позволит избежать побочных эффектов.

**Чувствительность к регистру**

В JavaScript все элементы, включая имена переменных, функций и операторов, чувствительны к регистру и должны всегда содержать одинаковые наборы прописных и строчных букв.

Например, ключевое слово while должно набираться как while, а не While или WHILE.

Аналогично num, NUM и Num – это три разные переменные.

**Комментарии**


©Pechora-PRO

Комментарии позволяют выделить фрагмент программы, который не выполняется интерпретатором JavaScript, а служит лишь для пояснений содержания программы.

Комментарии в JS могут быть однострочными и многострочными.

Однострочные комментарии начинаются с двойного слэша (//). Текст считается комментарием до конца строки:

```
<script>

    // объявляем переменную

    var num = 5;

    alert( num );  // выводим переменную

</script>
```

Многострочный комментарий начинается с слэша и звездочки (/*), а заканчивается ими же в обратном порядке (*/). Так можно закомментировать одну и более строк:

```
<script>

    /* многострочный комментарий

    эта часть кода не выполнится

    var num = 5;

    alert( num );

    */

    /* многострочный комментарий на одной строке */

    alert("Привет, Мир!");

</script>
```

## Идентификаторы

Идентификатор — это последовательность букв, цифр, символов подчёркивания (_) и знаков доллара ($). Цифра не может быть первым символом идентификатора, т. к. тогда


©Pechora-PRO

интерпретатору JavaScript труднее отличать идентификаторы от чисел.

Идентификаторы выступают в качестве имён переменных, функций, свойств объекта и т. д.

Примечание. Хотя это и не запрещено стандартом, использовать русские символы в именах идентификаторов не принято .

Исторически, программисты использовали разные способы объединения нескольких слов для записи идентификаторов. Сегодня есть два устоявшихся негласных стиля:

- camelCase
- snake_case.

Например:

```
<script>

    var userLogin = "master"; // стиль camelCase

    var element_id = 5;  // стиль snake_case

</script>
```

В JavaScript наиболее популярным стилем именования идентификаторов, состоящих из нескольких слов, является camelCase – "верблюжья" нотация.

**Примечание.** В JavaScript объединение нескольких слов для записи идентификаторов с применением дефисов запрещено. Они зарезервированы для математических вычитаний.

**Ключевые и зарезервированные слова**

Стандарт ECMAScript определяет набор ключевых слов (keywords), которые не могут использоваться в качестве идентификаторов. Зарезервированные слова имеют определенное значение в языке JavaScript, так как они


©Pechora-PRO

являются частью синтаксиса языка. Использование зарезервированных слов приведет к ошибке компиляции при загрузке скрипта.

Вот всего лишь некоторые зарезервированные ключевые слова: break, case, catch, class, while, continue, function...

## Курс Программирование в JavaScript

## Тема 1. Введение в JavaScript Урок 2. Переменные и константы

## Объявление переменной

JavaScript-сценарию обычно нужно работать с информацией.

Переменные используются для хранения этой информации и являются основным понятием любого языка программирования.

**Переменная** — это именованная область памяти, в которой мы можем хранить какие-либо данные, например, строки или числа. Переменные похожи на условный контейнер в памяти компьютера. Этим контейнерам можно присваивать имена и впоследствии использовать эти имена для извлечения и изменения хранимых данных.

Прежде чем использовать переменную в JavaScript, ее необходимо **объявить**.

### Объявление с помощью ключевого слова var

Для объявления или, другими словами, создания переменной используется ключевое слово var (от variable — переменная):

После объявления переменной ей можно присвоить некоторое значение, это называется инициализацией переменной:


©Pechora-PRO

```
var myLogin;

myLogin = "master"; // сохраним в переменной строку

// можно совместить объявление переменной и ее инициализацию

var age = 33; // сохраним в переменной число
```

Значение переменной сохраняется в соответствующей именованной области памяти и в дальнейшем доступ к этой области осуществляется при обращении по имени переменной. Если переменную не инициализировать, то ее начальное значение остается неопределенным undefined.

При желании можно объявить сразу несколько переменных, разделив их запятыми.

```
// объявление нескольких переменных

var email, pwd, role;
```

Можно совместить объявление и инициализацию нескольких переменных.

```
// объявление и инициализация нескольких переменных

var email = 'master@mail.ru', pwd = '12345', role = 'admin'
```

## Объявление с помощью ключевого слова let

Ключевое слово let появилось в ECMAScript 6, его, упрощённо, можно назвать блочной версией var.

```
<script>

    // в простых примерах ничего не меняется

    // объявляем и инициализируем переменную myLogin

    let myLogin = "master";

    // объявляем переменную age

    let age;

    // выводим значения переменных

    alert(myLogin); // выведет содержимое переменной
```


©Pechora-PRO

```
    alert(age); // выведет undefined
</script>
```

## Имена переменных

Имена переменных могут начинаться с одного из следующих символов:

- буква в верхнем или нижнем регистре: a-z, A-Z
- символ нижнего подчеркивания: _
- знак доллара: $

Вслед за первым символом имени переменной можно использовать последовательность букв, цифр и символа подчеркивания без пробелов и знаков препинания.

Поскольку JavaScript чувствителен к регистру, буквы включают символы от A до Z (верхний регистр) и символы от a до z (нижний регистр).

Примеры правильных имен переменных:

- var myName;
- var my_adress;
- var _x;
- var tel512_7456;

Примеры неправильных имен переменных:

- var 512group; // начало не может быть цифрой
- var my-adress; // дефис '-' не является разрешенным символом
- var my adress; // пробел не является разрешенным символом

При выборе имен переменных нельзя использовать ключевые слова, то есть слова, используемые в определениях конструкций языка. Например, нельзя выбирать слова var, return, class, const, true, false, function и ряд других.


©Pechora-PRO

Существуют проверенные опытом правила именования переменных:

- имена, состоящие из одной буквы, равно как и имена, не несущие смысловой нагрузки, можно применять только в том случае, если они используются в небольшом фрагменте кода и их применение очевидно;
- используйте составные имена из нескольких слов, чтобы как можно точнее описать назначение переменных.
- возможны два способа объединения слов для создания имени: использование "верблюжьей" нотации (camelCase) или запись слов через символ нижнего подчеркивания (snake_case).
- при выборе имен применяйте только английский язык, т.к. среди разработчиков всего мира принято использовать английский язык для имён переменных.


©Pechora-PRO